



# Scaling Horizons: Mastering LLM Applications with RAG and Fine-Tuning

In our time of rising popularity of Generative AI, we are facing a spike of approach usage and the urge to integrate it in a lot of existing applications. There are numerous tutorials and articles on that topic, but generally speaking, we can highlight three standard de facto approaches as of now:

1. **Base model utilization** – using existing ‘base’ models without any modifications, which (mostly) were trained on general data, like internet pages, Wikipedia, Stack Overflow, and so on.
2. **RAG** – Retrieval Augmented Generation. Technique to enrich models' knowledge with additional context. Usually utilizing vector or [embeddings](#), but not limited to that.
3. **Fine-tuning** – Model training at a small scale. Basically, it embeds external data inside of the model's ‘brain’.

With that – a few questions arise:

- How do we choose how much data we need for augmentation or fine-tuning?
- Is it time to go from ‘Base’ to ‘RAG’, or even ‘Fine-tuning’?
- How to make sure that we are progressing in the proper direction using the next step? Does it help?
- How do we choose where to stop?

In this article, we will define the RAG applications *scaling method* (how we are going to expand the model’s domain or task-specific knowledge) and try to supply the readers with a deterministic measurement tool on how to achieve our desired horizons based on the *evaluation* of LLM-based applications - testing their performance across various metrics to ensure they are accurate, fair, and reliable.



## LLM response evaluation

Before we dive directly into the scaling method, let us take a step back and look at the available metrics for evaluation. We want to show that deterministic approaches and overall [metric-driven-development](#) are the best reasoning tool for expanding the knowledge base of LLM-based applications (we will expand this idea in the next section).

### Metrics overview

When it comes to model evaluation, we need to understand what and how we can evaluate.

The ‘evaluation’ process itself – is an assessment of response correctness for the given model. The easiest way of doing this: a human evaluator that assesses response quality. This is often cumbersome and not productive. Aside from those little inconveniences – this is also slow, unreliable, and hard to automate and scale.

But let us evolve it further – how will humans assess response? Just “Good” or “Bad”? What if there are other values between these two?

Let us add a bit more determination and look at metrics that emerged in the field historically.

The “classic” Machine Learning metrics, which exist for a while to measure specific ML use-cases (mostly non-NLP related):

- Classification metrics:
  - Accuracy
  - Precision and Recall
  - F1 Score
  - ROC Curve and AUC
- Regression Metrics
  - Mean Absolute Error (MAE)
  - Mean Squared Error (MSE)
  - Root Mean Squared Error (RMSE)
  - R-squared (Coefficient of Determination)
- Clustering Metrics
  - Silhouette Score
  - Davies-Bouldin Index
- Multi-Class Classification Metrics
  - Confusion Matrix
  - Micro and Macro Averages



For NLP-related tasks let us consider the history of metrics evolution, as seen below (Figure 1):

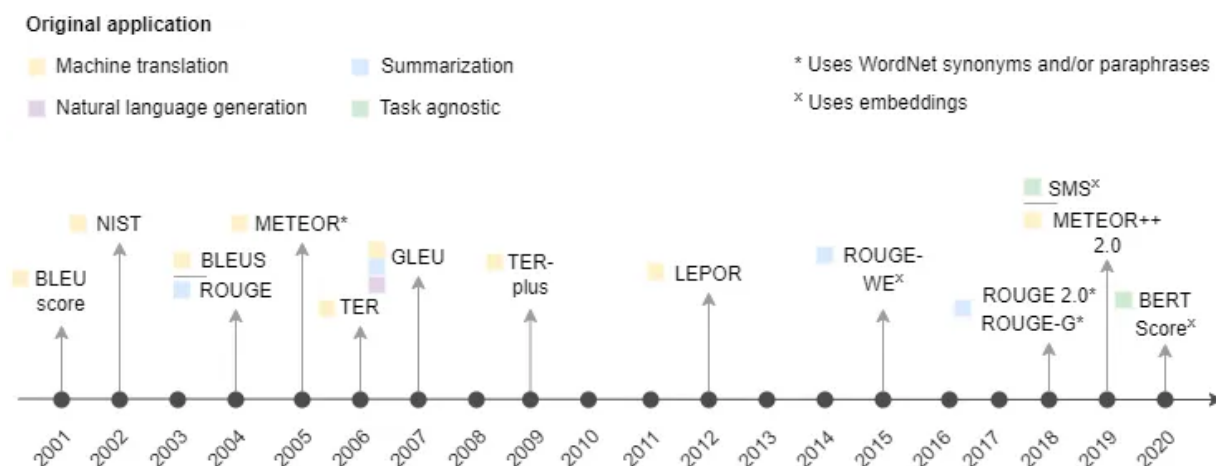


Figure 1: NLP (Natural Language Processing) metrics timeline ([source](#))

The traditional NLP metrics have common limitations that arise from the Generative AI evolution. They are limited by the context understanding with the appearance of ‘self-attention’ (see the [original whitepaper for more details](#)).

For example – BLEU score. BLEU (Bilingual Evaluation Understudy) focuses on comparing translation with high-quality references. This results in good precision of word matches but overlooks context and semantics ([reference](#)).

This is exactly the spot for us to transfer this information closer to our use case.

Since most LLM-based applications are NLP tools and work with text, they are performing the following tasks:

1. Summarization
2. Categorization
3. Text generation (including translation, chat, and question answering).

We would state that summarization and categorization can be measured using the [traditional ML metrics we briefly mentioned above](#), and overall fall into the category of simpler cases for measurement. But what about text generation?

Since question-answering applications with the use of Generative AI are those that have mostly open-ended questions, this is exactly the ‘complex’ case to measure. Let us look at what tools we have at our disposal for modern NLP evaluation:



1. New metrics – this is the set of modern (-ized) metrics to solve the limitations of ‘classic’ ones.
  - a. [BERTScore](#) - leverages the pre-trained contextual embeddings from BERT and matches words in candidate and reference sentences by cosine similarity.
  - b. [BLEURT](#) - BERT + BLEU metrics. Excellent example of solving the BLEU limitations with BERT score capabilities. Considers context and semantics.
  - c. [Entailment Score](#) – uses the concept of ‘natural language inference’ to calculate logical relationships within the text.
2. LLM-assisted evaluation – taking the idea of LLM evaluation and context understanding an emerging field of evaluation develops tools that use LLM to evaluate LLM. The most common ones are:
  - a. [GPTScore](#)
  - b. [G-Eval](#)

The core idea is that a generative pre-trained model will assign a higher probability of high-quality generated text following a given instruction and context.

We also should mention some of the ready-to-use LLM evaluators, mostly created on a proprietary basis, like [Vectara’s](#) HHEM ([Hallucination-evaluation Model](#)):

This model was trained using SentenceTransformers Cross-Encoder class. The model outputs a probability from 0 to 1, 0 being a hallucination and 1 being factually consistent. The predictions can be thresholded at 0.5 to predict whether a document is consistent with its source. ([citation reference](#))

LLM-based evaluation – is a novel approach, which on the one hand increases accuracy, but on the other hand, we might end up stuck in the “Evaluating the evaluation”.

## LLM Evaluation Frameworks

Fortunately, we are not the first ones to reason for all this. There are already good implementations that can be used for production systems. Let us consider a few examples

### 1. Langchain tools

Langchain is an orchestration framework used for the development of LLM-based applications. There is an excellent overview article published by Andrii Colodnitchii and Yuri Senko ([reference](#)) worth exploring if you are not familiar with this tool. The evaluation tools at our disposal are:



- a. LangChain Evaluators:
    - i. String Evaluators: These evaluators assess the predicted string for a given input, usually comparing it against a reference string.
    - ii. Trajectory Evaluators: These are used to evaluate the entire trajectory of agent actions.
    - iii. Comparison Evaluators: These evaluators are designed to compare predictions from two runs on a common input.
    - iv. Chain Comparisons: This example uses a comparison evaluator to predict the preferred output. It reviews ways to measure confidence intervals to select statistically significant differences in aggregate preference scores across different models or prompts.
  - b. [LangChain Benchmarks](#) – separate package for evaluation, heavily based on LangSmith (LangChain’s test & debug platform)
2. [LLama-index](#) – Llama-based RAG toolkit with embedded evaluation tools
- a. Correctness: Whether the generated answer matches that of the reference answer given the query (requires labels).
  - b. Semantic Similarity: Whether the predicted answer is semantically like the reference answer (requires labels).
  - c. Faithfulness: Evaluate if the answer is faithful to the retrieved contexts (in other words, whether there's hallucination).
  - d. Context Relevancy: Whether retrieved context is relevant to the query.
  - e. Answer Relevancy: Whether the generated answer is relevant to the query.
  - f. Guideline Adherence: Whether the predicted answer adheres to specific guidelines.
  - g. Question Generation: In addition to evaluating queries, LlamaIndex can also use your data to generate questions to evaluate on. This means you can automatically generate questions and run an evaluation pipeline to test if the LLM can answer questions accurately using your data.
3. [RAGAS](#) – e.g. - RAG Assessment library. Open-source, model agnostic approach for the RAG evaluation. RAG denotes a class of LLM applications that use external data to augment the LLM’s context. There are existing tools and frameworks that help you build these pipelines but evaluating it and quantifying your pipeline performance can be hard. This is where Ragas (RAG Assessment) comes in.



## RAG

### How to think about eval

| Ragas score                                                                     |                                                                                                       |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Generation                                                                      | Retrieval                                                                                             |
| How well the LLM answered the question                                          | How relevant the content retrieved is to the question                                                 |
| <b>Faithfulness</b><br>How factually accurate is the generated answer           | <b>Context precision</b><br>The signal to noise ratio of retrieved context                            |
| <b>Answer relevancy</b><br>How relevant is the generated answer to the question | <b>Context recall</b><br>Can it retrieve all the relevant information required to answer the question |

Figure 3. RAGAS approach ([source](#))

In the next parts of the article, we are going to use this framework to build our evaluation pipeline for these reasons:

1. It is completely open source
2. It is well-designed and lightweight, providing an excellent option to use in CI/CD pipeline

The main characteristics of this framework are:

- Ground truth – test set data which will be used as a reference for the evaluation. An example can be seen [here](#).
- Integrations – LangChain, LLama-index, Deep-eval, and others (pending)
- Hybrid evaluation – as noted in Figure 3, we are going to evaluate both generation and retrieval

And for sure other frameworks exist, like: – [Weights&Biases](#), [TruLens](#), [Deep-Eval](#), [Arthur Bench](#), [UpTrain](#) . (As a note - We have a plan of comparison of those frameworks in the next articles, so stay tuned!).



## Section outcome

Now that we have reviewed the existing metrics and approaches, let us develop a pipeline for evaluation. The types of practical applications differ, and the aim of this article is to explain how to move from a prompt to RAG and fine-tuning, and how to assess these processes. We will focus mainly on metrics for Chat (or Q&A) applications. We mentioned earlier that we could consider problems like summarization or categorization, which have their own unique metrics. However, evaluating open-ended questions is more challenging. Let us try to pull all this information together.

## Knowledge Scaling Method

### Conventional Development Flow

Overall LLM-based application development can be described in the following picture:

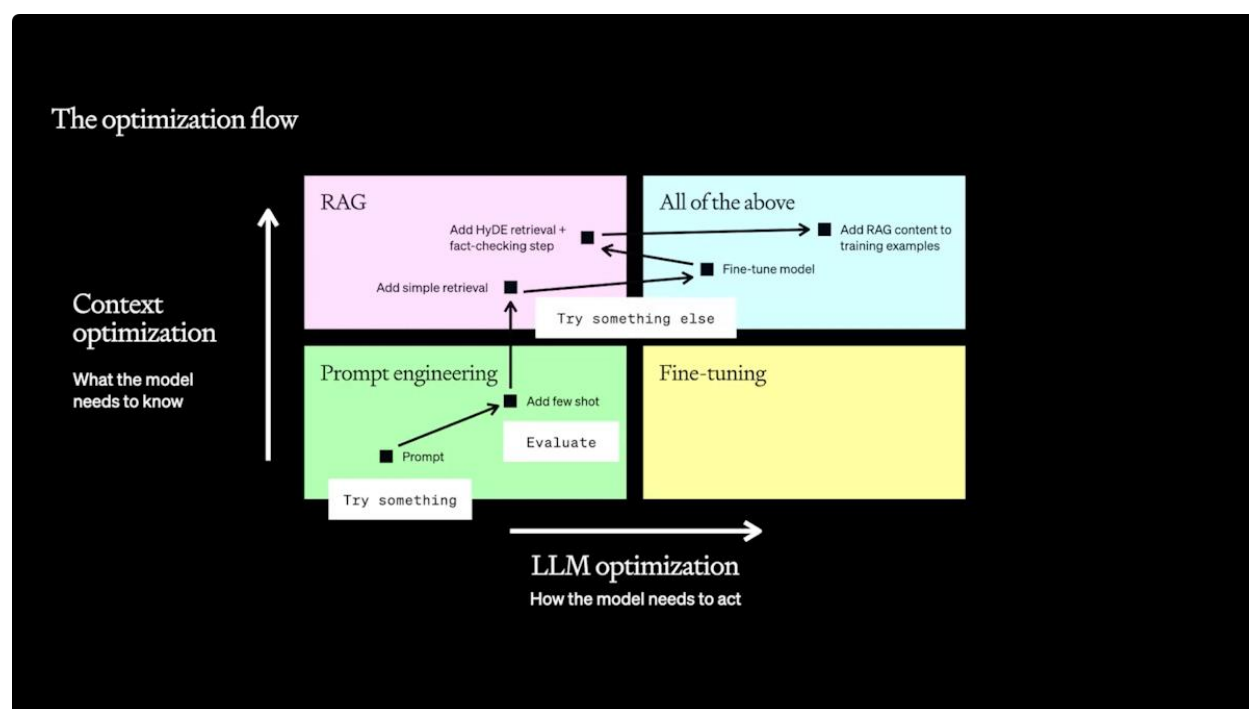


Figure 4.: LLM lifecycle ([source](#))

With an important note – the higher we are in this diagram on the top-right corner, the more expensive the application becomes. Prompt engineering is cheap, whereas fine-tuning is [really expensive](#).

But there is more – what about augmentation data? Let us say we have an existing 1TB database with data, and we want to make this data available for our model. Do we need to



prepare all our data to be available through embedding generation? There is no easy answer, but we are sure that this will take a bit of time and resources.

There are reference architectures for RAG implementations for major cloud providers ([Azure](#), [AWS](#), [GCP](#)) but only on the GCP's architecture we can spot something different:

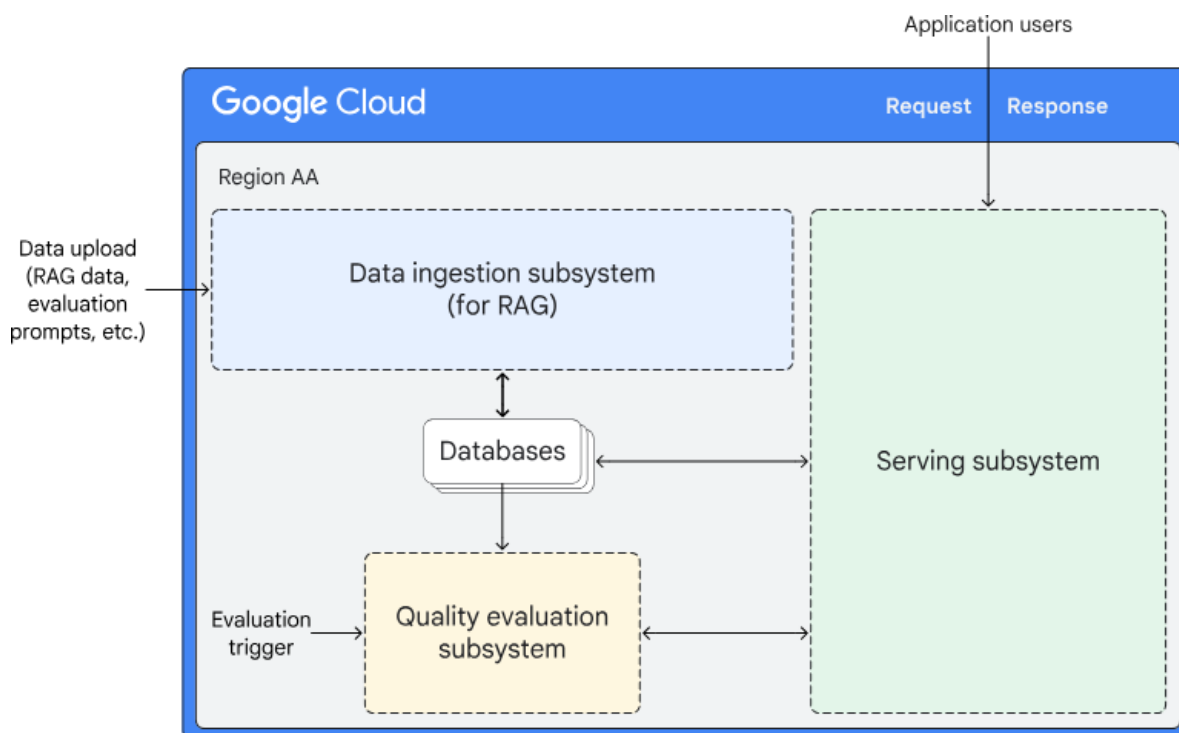


Figure 5: GCP RAG reference architecture ([source](#))

This architecture exposes an additional component – the ‘Quality evaluation subsystem’. The first impression – is, “yeah, this makes sense”. But after a closer look and after a previous section’s metrics – we can see this is all going.

*We would like to outline this component as crucial to the overall application success.*

In the next section, we will establish a firm foundation for further discussion by providing practical examples to enhance our understanding.

## Question-answering Example

In a chat-based application, a user can model ANY questions. Let us imagine we have a car repair service chatbot with access to service’s repair statistics. We surely can say that “What parts do I need to fix Toyota Corolla’s A/C” is more important for us than “Who won F1 in 1995?”. Questions are relevant, but these are extreme examples (still worth considering).

Let us look on a bit more nuanced difference:





1. “What parts do I need to fix Toyota Corolla’s A/C”
2. Vs.: “How often do I need to change refrigerant in A/C of Corolla according to maintenance statistics of last let’s say - 5 years”
3. Or even: “What compressor manufacturers are more reparable for Toyota Corolla’s A/C – Valeo or Denso, according to last repair statistics”

Couple of notes about these examples:

1. The base model will have a response for all of them (do not believe my words blindly – try asking ChatGPT)
2. The first question is the closest, but the answer will not consider car repair service statistics. In our practice, the most common case is “worn-out pipes”
3. The second and third answers will be even less precise.
4. The same way for humans – when someone asks us a question that requires data, we are not familiar with, then we need access to some external source of information to answer it. For instance, we could imagine that to answer question 2 we need to see all refrigerant changes in the last 5 years and take an average for that.
5. The same goes for the third question. This question is even more specific, requiring us to not only get repair statistics of A/C compressor replacement but to get repair statistics of specific manufacturers to provide this information for LLM for comparison.

*Fun fact:* The questions set is the real questions reported by the local car repair service owner.

Let us summarize our experience so far:

- Chat and general Q&A applications are the applications with open-ended questions.
- We will not restrict users' input (to a specific topic) since we do not want to significantly decrease the chat usefulness.
- We do not know what questions users will be asking, moreover – what questions and formats will be used frequently and what we need/want to cover.

## Agility

From the example and context data, we have observed in previous sections - we can make a couple of conclusions:

1. Everything is about data. What data we have, what we want to provide and especially - what end users need.
2. We do not exactly know what users need – we can just *assume* what they need.  
Referring to our Q/A example – the simplest question is the only thing they need, or will



everyone be asking complex comparison questions with specific details? We cannot be sure.

3. We better be as flexible as possible – delivery needs to be iterative and flexible.
4. We need to audit and keep track of questions which will help us understand on how to evolve knowledgebase (as we have mentioned – it is not feasible to index all data we have immediately, plus the more we are trying to cover – the more complex application becomes)

It reminds us of something familiar! Sounds exactly like agile.



Figure 6: Agile methodology ([reference](#))

## Scaling the Knowledge

As per Agile – starting small helps. As we have mentioned on GCP RAG reference architecture – ‘Quality evaluation’ needs to be built from day 0 to keep track of any direction of chat application evolution.

How should we approach this?

1. Form a [hypothesis](#).
  - a. Create a Ground truth dataset (for future RAGAS’ evaluation)
2. Build an application based on this hypothesis



3. Continuously audit:
  - a. Questions
  - b. Answers and their evaluation scores
  - c. Distinguish important questions from non-important, capture scores for improvement
4. Reason for new data and repeat!
  - a. Incoming questions from users can help us understand what the target knowledge we need to expose to our model.
  - b. Metrics can help us evaluate – as per [Metric-driven development](#), so we know if we are improving the model or not.
  - c. New questions can help us refine our Ground truth dataset, extending its quality
  - d. The importance of [good quality data is hard to over-state!](#)

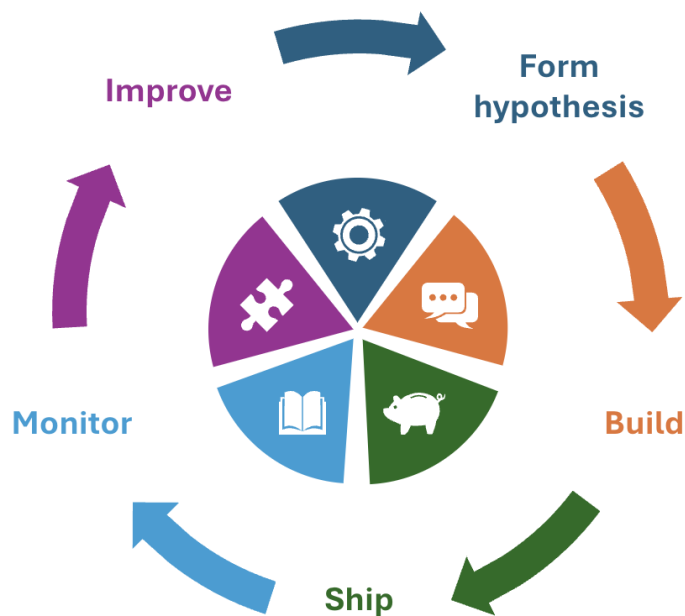


Figure 7: Pipeline methodology (agile inspired)

Let us review the example scope of the initial iteration:

1. Expose LLM application via service/user interface.
2. Build a monitoring/evaluation system



So, the next iteration scope is (the car repair service chatbot):

1. Assume that the majority of questions will be about one-dimensional knowledge (see Q&A example, question 1), requiring one entity (or document) indexing
2. Prepare Ground-truth dataset for evaluation based on hypothesis
3. Augment model with data
4. Deploy & monitor, verify hypotheses
5. Meanwhile, the amount of monitoring data will add up allowing us to plan on the next iterations

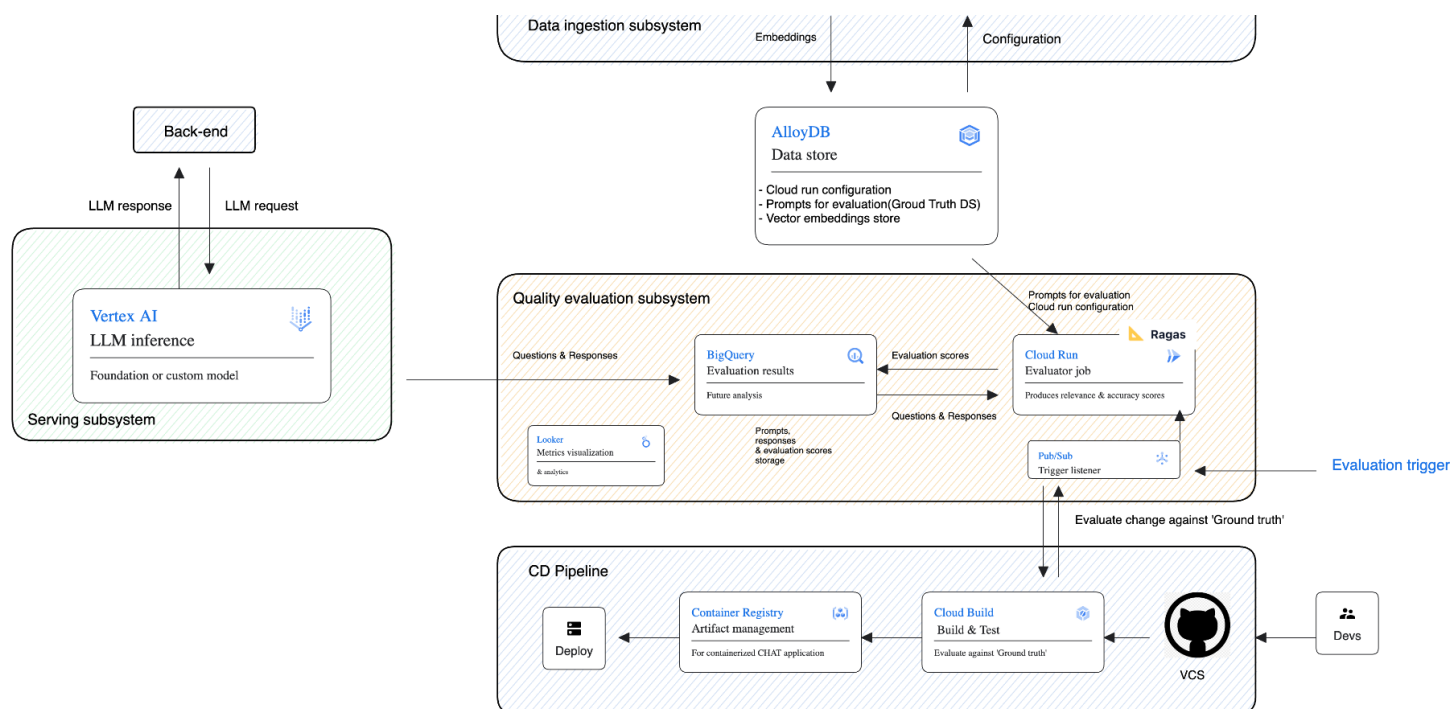


Figure 8: Evaluation subsystem details

## Unanswered questions

We have started the article with a set of questions, so as not to leave any stones unturned, let us review them.

**Q:** How do we choose how much data we need for augmentation?



**A:** As we have stated in the *Knowledge scaling method* section – start small, with one dimension of data for augmentation. Monitor your application to get more insights into where to move next and what entities or their relations to monitor.

**Q:** Is it time to go from ‘Base model’ usage to ‘RAG’ or even ‘Fine-tuning’?

**A:** As we have already stated - *prompt engineering* sets the tone and configuration of the model response but does not consider any external data. The next one is – base model. Base Model knows the language and data that was in public at the time of training, and that is it. Retrieval augmentation to fine-tuning – is a way more complex consideration, but as we have mentioned also, we need to have a Golden dataset (Ground truth) to train the model and have the best possible result. This is rarely easy in early application lifespan, since we must establish our ground truth only on our hypothesis, which may, or may not be correct. This needs to be verified before proceeding with a more complex approach (fine-tuning) too because fine-tuning requires a question-answering dataset. To summarize – start small, and consider more [complex cases only when you are sure they needed](#)

**Q:** How to make sure that we are progressing in the proper direction using the next step? Does it help?

**A:** Monitoring and evaluating pipelines will help with this. Build one and use it as a tool to measure progress.

**Q:** How do we choose where to stop?

**A:** We would say this is more a business question to consider, but having an evaluation pipeline in place, we can measure quantitative metrics on amount of covered and uncovered questions and establish a baseline (just like in a test coverage). Despite that we would outline that optimization of questions performance outside of domain we are working with is not the best option.

## Summary

The approach we have introduced will help us to deliver faster and iteratively, as in commonly used agile methodology. We also could improve application step by step. As a result – we will be able to make sure that we are evolving application in proper direction - extending knowledgebase for the cases we really care about.



#### About the Author:



#### **Rostyslav Volskyi**

Rostyslav Volskyi is a technical leader engineer at Capgemini Engineering Eastern Europe GEC with more than decade of experience with a strong back-end development background. AWS certified Developer & Solution Architect